Ludo Game Report

1 Data Structures Used to Represent Players and Pieces

1.1 Piece Structure

Each piece is represented by a structure containing three variables.

```
typedef struct {
    short location;
    short index;
    short captureCounter;
} piece;
```

location
This attribute is central to the piece's state within the game. It encodes the entire position logic of the piece, which can take the following values:

- -2: the piece is in the base.
- -1: the piece has reached home.
- 0 to 51: positions on the standard path.
- -10 to -5: positions in the home straight.

index
Identifies the piece among the four pieces belonging to the player, allowing easy access to and manipulation of a specific piece when needed.

captureCounter
Keeps track of how many captures the piece has made. This information is required when determining whether the piece can enter the home straight.

1.2 Player Structure

Each player is represented by a structure containing seven variables.

```
typedef struct {
    char  color[7];
    piece Piece[NUM_OF_PIECES];
    short piecesInPath;
    short piecesInBase;
    short piecesInHome;
    short startingPosition;
    short homeStraightEntery;
} player;
```

color
Represents the colour of the player's pieces.

`Piece`
: An array holding the four pieces that belong to the player. Each element is an instance of the `piece` structure, allowing each piece's state to be managed individually.

`piecesInPath`
: Tracks the number of pieces currently on the standard path, which helps in choosing which piece to move.

`piecesInBase`
: Tracks how many pieces are still in the base.

`piecesInHome`
: Counts how many pieces have reached home, which is essential for determining when a player has finished the game.

`startingPosition`
: Represents each player's starting cell, so that the structure can be generalised without considering which player is currently playing.

`homeStraightEntery`
: Represents each player's approach cell, again allowing the player structure to be generalised without considering which player is currently playing.

1.3 Justification for the Chosen Structures

The `player` structure encapsulates all attributes that define a player's state, making it easy to manage each player's pieces collectively. The nested `piece` array, in turn, allows each piece of a particular player to be accessed and updated separately.

By maintaining counts of pieces in each state (`piecesInPath`, `piecesInBase`, `piecesInHome`), the structure enables quick checks and updates without repeatedly iterating over the pieces array. In addition, by associating each player with a starting position and a home-straight entry cell, the same logic can be applied to every player, which keeps the code general.

2 Function Justification and Code Efficiency

1. `diceRoll()`

   - Generates a random number between 1 and 6, simulating a dice roll.
   - **Efficiency:** $O(1)$.

2. `nextplayer(int currentPlayer)`

   - Determines the next player in sequence, ensuring the game progresses in cyclic order (Red $\rightarrow$ Green $\rightarrow$ Yellow $\rightarrow$ Blue).
   - **Efficiency:** $O(1)$.

3. `InitializePlayer()`

   - Initialises the properties of each player, including their pieces and starting positions.
   - **Efficiency:** $O(n^2)$. However, this function is called only once per game, so it does not affect overall efficiency.

4. `EligibleToHomeStraight(int currentPosition, int expectedLocation, int currentPlayer)`

- Iterates from the current position to the expected location of a piece (`expectedLocation = currentPosition + diceRoll`) to determine whether the player's approach cell is passed during the move. If so, the piece enters the home straight instead of continuing along the standard path.
- **Efficiency:** $O(1)$, since the number of steps is bounded by the dice value.

5. `getPiecesInPath(int p, int pathPieces[])`
   - Returns an array containing the indexes of the pieces currently on the standard path. This is required to decide which piece to move when more than one piece is on the path. It is also used to identify which pieces are in the base, so that a piece can be moved to the starting cell when a 6 is rolled.
   - **Efficiency:** $O(n)$.

6. `positionMaker(int index, int player, int new_position)`
   - Sets the location of a piece that is about to enter the home straight from the standard path.
   - **Efficiency:** $O(1)$.

7. `updatePosition(int p)`
   - Iterates through each piece of the given player and recalculates the number of pieces in the base, on the path, and at home. This ensures that the game state remains accurate after every move.
   - **Efficiency:** $O(n)$.

8. `isCapture(int currentPlayer, int *capturedPlayer, int *capturedPiece)`
   - Takes the index of the current player so that only the pieces of the other three players are checked for capture. It also takes two pointers, to which it assigns the player index and piece index of the captured piece. This identifies which piece of which player must be returned to the base.
   - **Efficiency:** $O(n^2)$.

9. `reset(int capturedPlayer, int capturedPiece)`
   - Called after a capture to return the captured piece to the base of its player.
   - **Efficiency:** $O(1)$.

10. `movePiece(int currentPlayer, int roll)`
    - Handles all movement logic based on the dice roll, including capturing and entering the home straight. Encapsulating all movement-related logic in one function keeps the code modular and easy to maintain.
    - **Efficiency:** $O(n^2)$, as it must check multiple conditions.

11. `playGame(int firstPlayer)`
    - Takes the index of the starting player, which is determined in `main`. This is the central function that drives the game, controlling the flow of turns and checking for a winner.
    - **Efficiency:** The main loop runs until a winner is found, so its running time depends on the progress of the game.

3 Summary of Time Complexities

Function	Time Complexity
`diceRoll`	$O(1)$
`nextplayer`	$O(1)$
`InitializePlayer`	$O(n^2)$ (called once)
`EligibleToHomeStraight`	$O(1)$
`getPiecesInPath`	$O(n)$
`positionMaker`	$O(1)$
`updatePosition`	$O(n)$
`isCapture`	$O(n^2)$
`reset`	$O(1)$
`movePiece`	$O(n^2)$
`playGame`	Depends on game progress

Table 1: Time complexity of each function.